# Chapter 13

# Random Numbers and Stochastic Simulation

## 13.1 Stochastic Simulation

We have thus far considered only deterministic numerical methods for solving mathematical problems. An alternative approach, which is often very powerful for certain types of problems, is *stochastic simulation*. A serious study of stochastic simulation methods is beyond the scope of this book, but we will give a brief overview of these methods and the random number generators on which they depend.

Stochastic simulation methods attempt to mimic or replicate the behavior of a system by exploiting randomness to obtain a statistical sample of possible outcomes. Because of the randomness involved, simulation methods are also commonly known in some contexts as Monte Carlo methods. Such methods are useful for studying:

- Nondeterministic (stochastic) processes
- Deterministic systems that are too complicated to model analytically
- Deterministic problems whose high dimensionality makes standard discretizations infeasible (e.g., Monte Carlo integration; see Section 8.4.4)

The two main requirements for using stochastic simulation methods are:

- Knowledge of relevant probability distributions
- A supply of random numbers for making random choices

Knowledge of the relevant probability distributions depends on theoretical or empirical information about the physical system being simulated. As a simple example, in simulating a baseball game the known batting average of a player might determine the probability that the player gets a hit in a given turn at bat. A more

practical example is simulating the diffusion of particles (e.g., neutrons) through a medium (e.g., a shielding material). One must know the "cross section" or "mean free path," which are measures of the probability of a particle collision occurring, and also the probabilities of each possible outcome of a collision. The path of a single particle through the medium is then simulated by a sequence of random choices, each of which is weighted by the appropriate probability. By simulating a large number of such particle trajectories, the probability distribution of the overall results can be approximated, with the accuracy attained depending on the number of trials.

## 13.2 Randomness and Random Numbers

The concept of *randomness* is somewhat difficult to define. Physical processes that we usually think of as random, such as flipping a coin or tossing dice, are actually deterministic if we know enough about the equations governing their motion and the appropriate initial conditions. The distinction between deterministic and random behavior has become blurred, however, by such concepts as chaotic behavior of dynamical systems. Owing to their extreme sensitivity to initial conditions, the behavior of such systems can be unpredictable in practice even though it is deterministic in principle. For example, detailed weather predictions are impossible beyond approximately two weeks, even though we have a good understanding of the physical processes involved.

One way of characterizing the unpredictability that we associate with randomness is to say that a sequence of numbers is *random* if it has no shorter description than itself. In other words, there is no more economical way to convey the sequence than simply listing its members. Thus, for example, though each of the sequences $\{1, 2, 3, 4, 5\}$, $\{1, 1, 1, 1, 1\}$, and $\{4, 1, 5, 3, 2\}$ may be equally likely to occur, only the latter would be considered random. In some cases, even when variables are not really random, such as the arrival times and service times for a queue, they may be so complicated or imprecisely known that they are best treated as random variables, and the study of such systems is often tractable only by stochastic simulation methods.

In addition to unpredictability, another distinguishing characteristic of true randomness is a lack of *repeatability*: one would not expect the same long random sequence of numbers or coin tosses to occur twice. However, lack of repeatability could make testing algorithms or debugging computer programs difficult, if not impossible. Thus, there are advantages to generating random numbers by a repeatable process, but repeatability is a two-edged sword. The statistical significance of a stochastic simulation depends on the independence of the trials, which in turn depends on using *different* random sequences for each trial. In 1955, before computers were so common, the RAND Corporation published a book entitled *A Million Random Digits*. It was used in selecting random trials for experimental designs and simulations (and perhaps as bedtime reading for insomniacs?). It was soon realized, however, that if everyone always started on page one, then all trials and simulations by all the book's users would depend on the quirks of the same random

sequence. This generated much debate on how to select a random starting point in the table of random numbers.

## 13.3 Random Number Generators

Although random numbers were once supplied by physical processes or tables, they are now produced by computers. Computer algorithms for generating random numbers are in fact deterministic, although the sequence generated may *appear* random in that it exhibits no apparent pattern. However, an algorithm for generating random numbers is a short description of the sequence it yields, which therefore by definition is not truly random, so such a sequence is more accurately called *pseudorandom*. Although a pseudorandom sequence may appear random, it is in fact quite predictable and reproducible, which is important for debugging simulation programs and verifying results. Moreover, because only finitely many numbers are representable on a computer, any sequence must eventually repeat.

A good random number generator should have as many of the following properties as possible:

- *Random pattern.* It should pass statistical tests of randomness.
- *Long period.* It should go as long as possible before repeating.
- *Efficiency.* It should execute rapidly and require little storage, since many simulations require millions of random numbers.
- *Repeatability.* It should always produce the same sequence if started with the same initial conditions.
- *Portability.* It should run on different kinds of computers and be capable of producing the same sequence on each.

It is very difficult to satisfy all of these requirements in a single random number generator. For example, some random number generators in common use produce highly correlated sequences, which may become visually evident when consecutive pairs or triples of members of the sequence are plotted in space. This phenomenon has prompted the remark "random numbers fall mainly in the planes" [316] and can invalidate simulation results obtained using such a generator.

Early attempts at producing random number generators on computers often relied on highly complicated procedures whose very complexity was felt to ensure randomness. An example is the "midsquare" method, which squares each member of the sequence and takes the middle digits of the result as the next member of the sequence. The lack of a theoretical understanding of such methods proved disastrous, however, and it was soon recognized that simple methods with a well-understood theoretical basis are far preferable.

### 13.3.1 Congruential Generators

Congruential random number generators have the form

$$x_k = (ax_{k-1} + b) \pmod{M},$$

where $a$ and $b$ are given integers, the starting integer $x_0$ is known as the *seed*, and the integer $M$ is approximately (often equal to) the largest integer representable on a given computer.

The quality of such a generator depends on the choices of $a$ and $b$, and in any case its period cannot exceed $M$. It is possible to obtain a reasonably good random number generator using this method, but the values of $a$ and $b$ must be chosen *very* carefully. Many random number generators supplied with computer systems are of the congruential type, and some of them are notoriously poor.

A congruential generator produces random integers between 0 and $M - 1$. In order to produce random floating-point numbers, say uniformly distributed on the interval $[0, 1)$, the random integers must be divided by $M$ (but *not* using integer division!).

### 13.3.2 Fibonacci Generators

Alternative methods that produce floating-point random numbers on the interval $[0, 1)$ directly are the Fibonacci generators, which generate the new value as a difference, sum, or product of previous values. A typical example is the subtractive generator

$$x_k = x_{k-17} - x_{k-5}.$$

We say that this generator has *lags* of 17 and 5. Not surprisingly, the lags must be chosen carefully to produce a good subtractive generator. Note that such a formula may produce a negative result, in which case the usual remedy is to add 1 to get back into the interval $[0, 1)$.

A Fibonacci generator requires more storage than a congruential generator and also requires a special procedure to get started (analogous to a multistep method for solving an ODE). On the other hand, Fibonacci generators require no division to produce floating-point results, and well-designed Fibonacci generators have very good statistical properties. Another advantage of Fibonacci generators is that they can have a much longer period than congruential generators. The reason for this is that the repetition of a single member of the sequence does not entail that all subsequent members will also repeat in the same order.

### 13.3.3 Nonuniform Distributions

Thus far we have discussed generating random numbers only from a uniform distribution on the interval $[0, 1)$. If we need a uniform distribution on some other interval $[a, b)$, then we can simply modify the values $x_k$ generated on $[0, 1)$ by the transformation $(b - a)x_k + a$ to obtain random numbers that are uniformly distributed on the desired interval.

A more difficult problem is to sample from nonuniform distributions. If the cumulative distribution function of the desired probability density function is easily invertible, then we can generate random samples with the desired distribution by generating uniform random numbers and then inverting them. For example, the

exponential distribution with rate $\lambda$ has the density function

$$f(t) = \lambda e^{-\lambda t}, \qquad t > 0,$$

and cumulative distribution function

$$F(x) = \int_0^x f(t)\,dt = 1 - e^{-\lambda x}.$$

Given $y = F(x)$, we can easily solve for $x$, obtaining

$$x = -\frac{\log(1-y)}{\lambda}.$$

Hence, to sample from the exponential distribution, we can take

$$x_k = -\frac{\log(y_k)}{\lambda},$$

where $y_k$ is uniform on $[0, 1)$.

Unfortunately, many important distributions are not easily invertible, and special methods must be employed to generate random numbers efficiently for these distributions. An important example is the generation of random numbers that are normally distributed (i.e., Gaussian) with a given mean and variance. Some methods for generating normally distributed random numbers are explored in Computer Problem 13.8. Available routines for normal random numbers often assume a mean of 0 and variance of 1. If some other mean $\mu$ and variance $\sigma^2$ are desired, then each value $x_k$ produced by the routine can be modified by the transformation $\sigma x_k + \mu$ to achieve the desired normal distribution.

## 13.4 Quasi-Random Sequences

Despite the quest to develop algorithms for generating perfectly random numbers, true randomness is not always a virtue. For some applications, such as Monte Carlo integration, achieving reasonably uniform coverage of the sampled volume can be more important than whether the sample points are truly random. The problem is that random sequences tend to exhibit random clumping (lightning may indeed strike in nearly the same place twice if it is truly random), leading to rather uneven coverage of the sampled volume for a given number of points. At the other extreme, perfectly uniform coverage can be achieved by using a regular grid of sample points, but as we saw in Section 8.4.4, such an approach does not scale well to higher dimensions.

A compromise between these extremes of coverage and randomness is provided by *quasi-random sequences*. Such sequences are in fact not random at all but are carefully constructed to give uniform coverage of the sampled volume while maintaining a reasonably random appearance. By design the points tend to avoid each other, so the clumping associated with true randomness is eliminated. The differences among these three sampling techniques are illustrated in Fig. 13.1. Such

quasi-random sequences, also called *low-discrepancy sequences*, are finding increasing use in Monte Carlo integration and other applications, such as optimization by random search, where uniform coverage of the sampled volume is more important than the statistical properties of the sampling procedure.

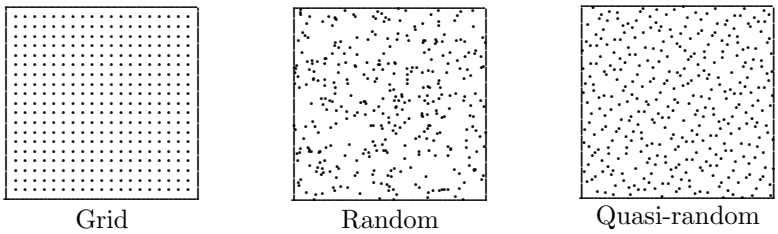


Figure 13.1: Three methods for sampling the unit square using the same number of points.

## 13.5 Software for Generating Random Numbers

Table 13.1 is a list of some of the random number generators available in standard software libraries. Generators are listed only for uniform and normal distributions. Some of these sources contain generators for several additional distributions, both continuous and discrete. Software for generating quasi-random sequences is available from `NR` [377, Section 7.7] and `TOMS (#647, #659, #738)`. Although stochastic simulation programs can be written in any general-purpose programming language, the particular needs of this field have motivated the development of special-purpose languages and software systems for simulation, including `GPSS`, `Simscript`, `Simula`, and `SLAM`.

## 13.6 Historical Notes and Further Reading

Monte Carlo methods were developed in the 1940s by von Neumann, Ulam, Metropolis, and others, primarily motivated by problems in nuclear physics such as neutron transport. For a philosophical discussion of the concept of randomness, see [33]. For a brief overview of randomness, random number generators, and simulation, see [203, 350]. For a detailed treatment of stochastic simulation and Monte Carlo methods, see [16, 49, 91, 113, 143, 144, 145, 223, 264, 293, 307, 394, 419]. Congruential random number generators were proposed by Lehmer in 1948. Volume 2 of the series by Knuth [279] contains a detailed discussion of random number generators and their analysis; see also [192]. For a survey of random number generators, see [10, 180, 252, 301, 364]. For Gaussian random number generators, see [453]. The development of quasi-random sequences dates to the work of Halton and of Sobol' in the 1960s. For a review of quasi-random sequences and their applications, see [349].

| Source | Uniform distribution | Normal distribution |
|---|---|---|
| FMM [152] | urand | |
| GSL | gsl_rng_uniform | gsl_ran_gaussian |
| HSL | fa04 | fa05 |
| IMSL | rnunf | rnnof |
| KMN [262] | uni | rnor |
| MATLAB | rand | randn |
| NAG | g05caf | g05ddf |
| Netlib | zufall | normalen |
| NR [377] | ran1/ran2/ran3 | gasdev |
| NumPy | random.rand | random.randn |
| NumPy | random.uniform | random.normal |
| RANLIB | ranf/genunf | gennor |
| SLATEC | rand/runif | rgauss |
| TOMS | sprng(#806) | grand(#488)/randn(#712) |

Table 13.1: Software for generating random numbers

## Review Questions

**13.1.** True or false: Stochastic simulation methods are useful only for nondeterministic systems.

**13.2.** True or false: A very complicated algorithm for generating random numbers tends to be better than a simple one.

**13.3.** What two requirements are essential for stochastic simulation methods?

**13.4.** In using stochastic simulation methods, why is repeatability desirable in practice, even if undesirable in theory?

**13.5.** List at least three desirable properties of a good random number generator.

**13.6.** What is meant by a *congruential* random number generator?

**13.7.** What is meant by a *Fibonacci* random number generator?

**13.8.** List two advantages and two disadvantages of a Fibonacci generator compared with a congruential generator for generating random floating-point numbers uniformly distributed on the interval $[0, 1)$.

**13.9.** If you have a random number generator for a uniform distribution on $[0, 1)$, how can you use it to generate random numbers uniformly distributed on the interval $[a, b)$ for given real numbers $a$ and $b$?

**13.10.** If you have a random number generator for a normal distribution with mean 0 and variance 1, how can you use it to generate random numbers uniformly normally distributed with mean $\mu$ and variance $\sigma^2$?

**13.11.** Why are *quasi-random* sequences sometimes more useful than truly random (or pseudo-random) sequences?

## Exercises

**13.1.** Consider a congruential random number generator

$$x_k = (ax_{k-1} + b) \pmod M,$$

with $b = 0$, $M = 8192$, and the seed $x_0 = 1$.

($a$) What is the period of this generator if $a = 2$?

($b$) What is the period if $a = 125$?

($c$) What is the longest possible period, given these values for $b$, $M$, and $x_0$?

**13.2.** ($a$) Choose a well-structured sport, such as baseball or football, and discuss how you would simulate a typical game. What type of information would be required to perform a fairly detailed and realistic simulation? Where would random choices enter into the simulation?

($b$) Would it be feasible to simulate a sport like basketball, soccer, or hockey in this manner? Try to characterize the differences from the previous case.

## Computer Problems

**13.1.** ($a$) In this exercise we will perform a *chi-square test* of the randomness of the sequence generated by a random number generator. Divide the interval $[0, 1)$ into $n$ equal subintervals. Choose a library routine for generating uniformly distributed random numbers on $[0, 1)$. Generate $k$ random numbers and count the number $n_i$ that fall into each subinterval, $i = 1, \ldots, n$. Compute the chi-square statistic

$$\chi^2 = \sum_{i=1}^{n} \frac{(n_i - k/n)^2}{k/n}.$$

In a table for the chi-square distribution (see [521], for example, or a book on statistics), look up the probability that the chi-square statistic has the value you obtained for $n - 1$ degrees of freedom. Carry out this experiment for $n = 10$ and 20, and for $k = 100, 200, \ldots, 1000$.

($b$) Implement your own random number generator, using either the congruential or Fibonacci method, and again perform the chi-square test as outlined in part $a$. Experiment with various choices for the parameters of the generator to see what effect these have on the randomness test. Can you find choices of parameters that produce particularly good or bad results in the randomness test (without being obviously bad, such as having a very short period)?

**13.2.** Monte Carlo integration requires random sampling of a region in $n$-dimensional space. If the sequence of random numbers one is using exhibits any serial correlation, then the sampling may systematically miss a portion of the region and may possibly even be confined to a subregion of lower dimension, which would obviously make the estimate for the integral erroneous. One way to detect such serial correlation is to plot pairs or triples of consecutive random numbers in two or three dimensions, so that any nonrandom pattern becomes apparent visually.

($a$) Use the congruential random number generator

$$x_k = (ax_{k-1} + b)(\text{mod } M),$$

with $a = 125$, $b = 0$, $M = 8192$, and the seed $x_0 = 1$, to generate a sequence of random integers, and convert each to a floating-point number $f_k$ on $[0, 1)$ by taking $f_k = x_k/M$. Plot 100 pairs of consecutive members of the sequence, i.e., $(f_1, f_2)$, $(f_3, f_4), \ldots, (f_{199}, f_{200})$, as points in the unit square. Do you notice any obvious pattern? Make a second plot using 1000 pairs. Now do you notice any obvious pattern?

($b$) Repeat the previous experiment, but try various values for the parameters $a$, $b$, and $M$. Can you find values that show no obvious pattern when plotted, even when you increase the number of points further?

($c$) Repeat the same experiment again, this time using a library routine for generating uniform random numbers on $[0, 1)$.

**13.3.** For each random number generator used in the previous exercise, generate a sequence of $n$ random numbers, where $n$ is a power of two. Use an FFT routine to compute the DFT of each sequence (see Chapter 12). In each case, does the transformed sequence appear to be biased toward any particular frequencies, or does it appear to be "white noise," with all frequencies represented approximately equally? You may need to use a fairly large value of $n$, say, $n = 1024$, to draw any significant conclusions. How do the results of this "Fourier test" compare with your other tests (i.e., do the same generators fare well or poorly)?

**13.4.** A sequence of random numbers distributed uniformly on $[0, 1)$ should have mean

$$\mu = \int_0^1 x\, dx = \frac{1}{2}$$

and variance

$$\sigma^2 = \int_0^1 (x-\mu)^2\,dx = \frac{1}{12},$$

where $\sigma$ is the standard deviation. Check each of the random number generators used in the previous exercise to see how close they come to these values for a sequence of length 1000. Do all of the generators pass this test?

**13.5.** ($a$) Suppose that you are given five independent random decimal digits. The possible outcomes can be classified much like poker hands. Show analytically that the probability of each possible outcome is given by the following table:

| Hand | Pattern | Probability |
|---|---|---|
| Five of a kind | `aaaaa` | 0.0001 |
| Four of a kind | `aaaab` | 0.0045 |
| Full house | `aaabb` | 0.0090 |
| Three of a kind | `aaabc` | 0.0720 |
| Two pairs | `aabbc` | 0.1080 |
| One pair | `aabcd` | 0.5040 |
| Bust | `abcde` | 0.3024 |

($b$) Use a library routine for generating random numbers to determine the probability of each possible outcome by random sampling. Experiment with various numbers of trials. How do your results compare with the analytical results in part $a$? What is the effect of taking more trials?

($c$) The ability to match the correct theoretical distribution of poker hands (called the *poker test*) is sometimes used as a measure of quality of random number generators. Try a poor random number generator from one of the previous exercises and see if it passes this test, for the same number of trials used in part $b$.

**13.6.** The *birthday paradox*: Use random trials to determine the smallest number of persons required for the probability to be greater than 0.5 that two persons in a group have the same birthday. Also try to justify your result analytically.

**13.7.** Use random sampling to determine the probability that the quadratic equation

$$ax^2 + bx + c = 0$$

will have only real roots, if each of its real coefficients $a$, $b$, and $c$ is randomly chosen from the interval $[-1, 1]$. Also try to justify your result analytically.

**13.8.** In this exercise, we will consider three different ways of generating random numbers that are normally distributed with zero mean and unit variance.

($a$) According to the *Central Limit Theorem*, if $x_k$, $k = 1, \ldots, n$ is a sequence of independent random numbers from a distribution with mean $\mu$ and variance $\sigma^2$, then as $n$ increases,

$$y = \frac{\sum_{k=1}^n x_k - n\mu}{\sigma\sqrt{n}}$$

approaches a normal distribution with a mean of 0 and a variance of 1. If the $x_k$ are uniformly distributed on $[0, 1)$, then $\mu = 1/2$ and $\sigma^2 = 1/12$, which implies that

$$y = \frac{\sum_{k=1}^n x_k - n/2}{\sqrt{n/12}}$$

approaches a normal distribution with a mean of 0 and a variance of 1. The choice $n = 12$ makes the foregoing formula particularly simple:

$$y = \sum_{k=1}^{12} x_k - 6,$$

i.e., we just sum up a dozen uniform random numbers and subtract 6, and the result should be approximately normally distributed with a mean of 0 and a variance of 1. You might also experiment with other values for $n$ to examine the tradeoff between cost and accuracy.

($b$) Generate two random numbers $x_1$ and $x_2$, uniformly distributed on $[0, 1)$. Then it can be shown that both

$$y_1 = \sin(2\pi x_1)\sqrt{-2\log(x_2)}$$

and

$$y_2 = \cos(2\pi x_1)\sqrt{-2\log(x_2)}$$

are normally distributed with a mean of 0 and a variance of 1.

($c$) Generate two random numbers $x_1$ and $x_2$ uniformly distributed on $[-1, 1)$, and let $r = x_1^2 + x_2^2$. If $r > 1$, reject this pair and generate a new uniform pair $x_1$ and $x_2$. If $r \le 1$, then it can be shown that both

$$y_1 = x_1\sqrt{-2\log(r)/r}$$

and

$$y_2 = x_2\sqrt{-2\log(r)/r}$$

are normally distributed with a mean of 0 and a variance of 1.

Implement each of these three methods for generating normally distributed random numbers. To test each routine, generate $n = 1000$ normal random numbers, compute a frequency histogram with bin size $\sigma = 1$, and compare the values obtained with the known values for a true normal distribution: $\ldots$, 2.14%, 13.59%, 34.13%, 34.13%, 13.59%, 2.14%, $\ldots$. Also compare your results with a histogram similarly obtained using a library routine for generating normally distributed random numbers. You may find it instructive to plot the histograms, perhaps superimposed with a suitably scaled normal density curve. Compare the methods with respect to accuracy and efficiency. You may also wish to experiment with other values for $n$.

**13.9.** One of the earliest examples of using randomness to compute a deterministic quantity is the *Buffon needle problem*, in which an approximate value for $\pi$ is determined by repeatedly throwing a needle onto a plane surface ruled with equally spaced parallel lines, such as the seams in a hardwood floor. We introduce an $(x, y)$ coordinate system to describe the position of the center of the needle, with the $x$ axis parallel to the lines. Clearly, the probability that the needle intersects a line is independent of $x$, and depends only on $y$ and the angle $\theta$ between the needle and the $x$ axis. For simplicity, assume that the length of the needle, as well as the distance between any two adjacent lines, is 1.

By symmetry, we need be concerned only with values for $\theta$ in the interval $[0, \pi/2)$, and for $y$ in the interval $[0, \frac{1}{2})$. To determine the proportion of this rectangle in $(\theta, y)$ for which the needle intersects a line, we choose random values for $\theta$ and $y$, uniformly distributed on $[0, \pi/2)$ and $[0, \frac{1}{2})$, respectively. Now a needle with its center at $(x, y)$ will intersect a line precisely when $y \leq \sin(\theta)/2$. Therefore, the probability that the needle will intersect a line is simply the ratio of the area under the curve $\sin(\theta)/2$ over the interval $[0, \pi/2]$ to the total area of the rectangle, which is just $\pi/4$. Thus, the approximate value for $\pi$ will simply be $2n/m$, where $n$ is the total number of trials and $m$ is the number of trials for which $y \leq \sin(\theta)/2$.

Use a library routine for generating uniform random numbers to implement this method for approximating $\pi$. How many trials are required to obtain $k$ correct digits of $\pi$, where $k = 1, 2, 3, 4$?

**13.10.** ($a$) Use the Monte Carlo method to compute the area of a circle of diameter 1 inscribed in the unit square, $0 \leq x, y \leq 1$. Generate a sequence $(x_k, y_k)$ of $n$ pairs of random numbers uniformly distributed on $[0, 1)$, and count the number $m$ falling within the circle. Since the area of the square is 1, the area of the circle should be approximated by $m/n$. Compare your result with the known area, which is given by $A = \pi r^2 = \pi/4$. Use various values for $n$, and try to characterize the size of the error as a function of the number of random trials.

($b$) Use the Monte Carlo method to compute the volume of a sphere of diameter 1 inscribed in the unit cube, $0 \leq x, y, z \leq 1$. Generate a sequence $(x_k, y_k, z_k)$ of $n$ triples of random numbers uniformly distributed on $[0, 1)$, and count the number $m$ falling within the sphere. Since the volume of the cube is 1, the volume of the sphere should be approximated by $m/n$. Compare your result with the known volume, which is given by $V = 4\pi r^3/3 = \pi/6$. Use various values for $n$, and try to characterize the size of the error as a function of the number of random trials.

($c$) Compare your results for parts $a$ and $b$ in terms of the accuracy attained as a function of the number of trials. Explain any relationship you may observe.

**13.11.** Use random trials to simulate the behavior of the Markov chain given in Computer Problem 4.12. For the same transition probabilities and initial state given there, perform $m$ trials, taking $k$ steps for each trial. Experiment with the values for $m$ and $k$, and compare the resulting probability distribution vectors with the results obtained by the methods in Chapter 4.

**13.12.** ($a$) Radioactive decay is an inherently nondeterministic process that can be simulated very naturally using the Monte Carlo method. Suppose that the probability of any given atom decaying over a time interval $\Delta t$ is given by $\lambda$, $0 < \lambda < 1$. Then the history of a single atom can be simulated by choosing a sequence of random numbers $x_k$, $k = 1, \ldots$, uniformly distributed on $[0, 1)$. The atom survives until the first value $x_k$ for which $x_k < \lambda$. Use this approach to simulate

an ensemble of $n$ atoms. Make a plot of the number of atoms remaining after $k$ time increments, $k = 1, \ldots, m$. Take $\lambda = 0.1$ and $m = 50$, and try the values $n = 10$, 100, 1000, and 10,000, making a separate graph for each value of $n$. How does the value of $n$ affect the smoothness of the resulting curve? Experiment with other values for $\lambda$, $n$, and $m$ to see the range of possible behaviors.

($b$) We can also develop a continuous, deterministic model of radioactive decay for an ensemble of atoms. The change in the number of atoms $\Delta n$ over time interval $\Delta t$ is proportional to the number of atoms present and the length of the time interval. Specifically,

$$\Delta n(t) = -\lambda n(t)\,\Delta t.$$

In the limit, this gives the ODE

$$n'(t) = -\lambda n(t),$$

whose solution is

$$n(t) = n(0)e^{-\lambda t},$$

the familiar decaying exponential model for radioactivity. For the same values of $\lambda$ and initial number of atoms $n$ as in part $a$, plot the resulting solution for $t = 1, \ldots, m$ on the same graph with the results from part $a$. How do the two models compare? How is their agreement affected by the initial number of atoms $n$? Which model do you think is closer to the way nature actually behaves? Why?

**13.13.** A *random walk* is a sequence of fixed-length steps, each taken in a randomly (and independently) chosen direction. A random walk can be used to model various physical phenomena, such as Brownian motion of small particles suspended in a fluid. In this exercise we will consider only the one-dimensional case in which the walk begins at 0. The walk proceeds by successive steps of unit length, with the direction—positive or negative—chosen randomly with equal probability. Write a program to implement a random walk of $n$ steps, using a uniform random number generator to choose the direction for each step, and compute the distance from the origin at the end of the walk. Run your program for $n = 10$, 100, 1000, and 10,000, averaging the distance from the origin at the end of the walk over several trials for each value of $n$. Characterize the average distance from the origin as a function of $n$.

**13.14.** In a two-dimensional random walk, steps are taken randomly on a two-dimensional lattice of points, with the direction—up, down, left, or right—chosen randomly with equal probability. It turns out that such a random walk can be used to solve the Laplace equation. Impose a grid of points on the domain of the equation, including the boundary, then start $n$ trials of a random walk from a given interior grid point, terminating each trial when some boundary point of the domain is hit. The solution at the starting grid point is then given by the average over all the trials of the solution values at the boundary points that were hit.

Implement this two-dimensional random walk approach to solve the Laplace equation, using the same grid and boundary values as in Example 11.5. Compute the solution at each of the interior grid points by averaging over $n$ trials from each point. How many trials are required to obtain results comparable to those in Example 11.5?

Because of the large number of trials required, the random walk method is not competitive with other numerical methods for computing the solution over the entire domain. The random walk method does have the advantage, however, that it can compute the solution at a single point without having to compute it anywhere else, which is in contrast with most other numerical methods. Moreover, the random walk method can easily accommodate very complicated boundaries that may be difficult to deal with using conventional numerical methods.

**13.15.** A polymer can be modeled as a sequence of $n$ segments (monomers) of fixed length joined by $n - 1$ universal joints, so that each angle between adjacent segments can take any value from 0 to 360 degrees with equal probability.

($a$) Use a uniform random number generator to implement this model of a polymer in two dimensions. Assume that the units chosen are such that the length of each segment is 1. The $n-1$ angles $\theta_k$ between successive segments should be uniformly distributed on the interval $[0, 2\pi)$. Try several different values for $n$, say 10, 100, or more if your computer budget allows (real polymers may have thousands of segments). Make several trials for each value of $n$ and compute the average overall length of the polymer (i.e., the distance between the endpoints of the first and last segments

in Cartesian coordinates). Try to characterize the average length of the polymer as a function of $n$. You may find it interesting to draw a picture of the polymer in two dimensions. Note that the result is simply a kind of random walk.

($b$) Repeat part $a$, but this time in three dimensions. To generate the direction of each new segment, you will probably find it most convenient to use spherical coordinates, with both the azimuth and zenith angles, $\theta$ and $\phi$, chosen as random numbers uniformly distributed on the interval $[0, 2\pi)$. To draw a picture and compute the length of the polymer, however, you will find it easiest to use three-dimensional Cartesian coordinates. Again, run several trials for each value of $n$ and try to characterize the average length of the polymer as a function of $n$. Does the increase in dimensionality make a significant difference in the length of the polymer for the same number of segments?

($c$) The previous model is unrealistic in part because chemical bonds are not universal joints, and they prefer certain bond angles over others. Assume that instead we have swivel joints in three dimensions, with a fixed angle $\theta$ and free angle $\phi$. Use this new model to simulate polyethylene, which has a favored bond angle of $\theta = 109°$. Again, run several trials for the same values of $n$ and characterize the average length of the polymer as a function of $n$. Does this model give significantly different results from the previous model? When plotted in three dimensions, do the simulated polymers given by the two models look appreciably different?

**13.16.** Suppose that neutrons enter one side of a lead shielding wall that is four units thick. Each neutron enters at a random angle and travels a distance of one unit before colliding with a lead nucleus. Such a collision causes the neutron to bounce off in a random direction to travel another unit distance before another collision, and so on. After eight collisions, a given neutron has lost all of its energy and therefore stops. Use Monte Carlo simulation to determine the percentage of neutrons that exit from the other side of the wall. Does it make a significant difference whether the wall is modeled as two-dimensional or three-dimensional?

**13.17.** Repeat one or more of the preceding exercises using a quasi-random sequence generator (see Section 13.5 for appropriate software) instead of a standard random number generator. Does the quasi-random sequence pass any of the randomness tests given in the first few exercises? Does the quasi-random sequence make a significant difference in accuracy or efficiency for any of your simulation results?

**13.18.** Evaluate the double integral given in Computer Problem 8.14 using Monte Carlo integration. Try both a standard uniform random number generator and a quasi-random sequence generator and compare their efficiency in attaining a given level of accuracy.